

Test Sheet

ROS2 CONTROL PIPELINE FOR AN INDUSTRIAL MANIPULATOR

Introduction

1.1 Purpose

The following tests were defined to verify the functionality of the developed system, study the user experience, and identify faults and weaknesses. The tests were selected to cover both general system integration and subsystem-specific requirements.

While the main purpose of this test sheet is to verify the functionality of the system, since the goal of this project was to develop a control pipeline for the robot manipulator which could be used in future research, it is important to observe and note the user experience of the tester when interacting with the system while verifying its functionality.

1.2 Equipment List

Table 1: Hardware

Hardware		
Equipment	Description	Number
Franka Research 3	Franka FR3 robotic manipulator	1
FR3 Control Box	Robot control unit for the Franka FR3 arm	1
Franka Emika Emergency Stop Device	Emergency stop for the Franka robot	1
Sintef Lab computer	Computer connected to the Franka control unit	1
Gripper	Gripper developed by Jakob Ingvar Utne Midtun for the RI Seaweed project	1
Raspberry Pi 4B	The MCU of the ROS2 system	1
Intel RealSense D455	Depth camera used for machine vision	1
Model cylinder	Cylinder model meant to simulate the seaweed growth cylinders	1
Laptop	Laptop with ROS2 installed to connect to the ros2 network of the robot	1

Table 2: Software

Software		
Program	Producent	Version
Ubuntu	Canonical	24.04
ROS2	N/A (Open Source)	Jazzy
realsense-ros	RealSenseAI (Open Source)	4.57.7
franka_ros2	Frankarobotics (Open Source)	2.4.0
franka_description	Frankarobotics (Open Source)	2.7.1
kuka_robot_descriptions	Kroshu (Open Source)	Astonishing Ampere
Arduino IDE	Arduino	1.8.19
Visual Studio Code	Microsoft	1.120

1.3 Setup

The procedures described in this test sheet are to be carried out using the Franka Research 3 workstation at the SINTEF MARO laboratory. The connection between the main system components is shown in the figure below.

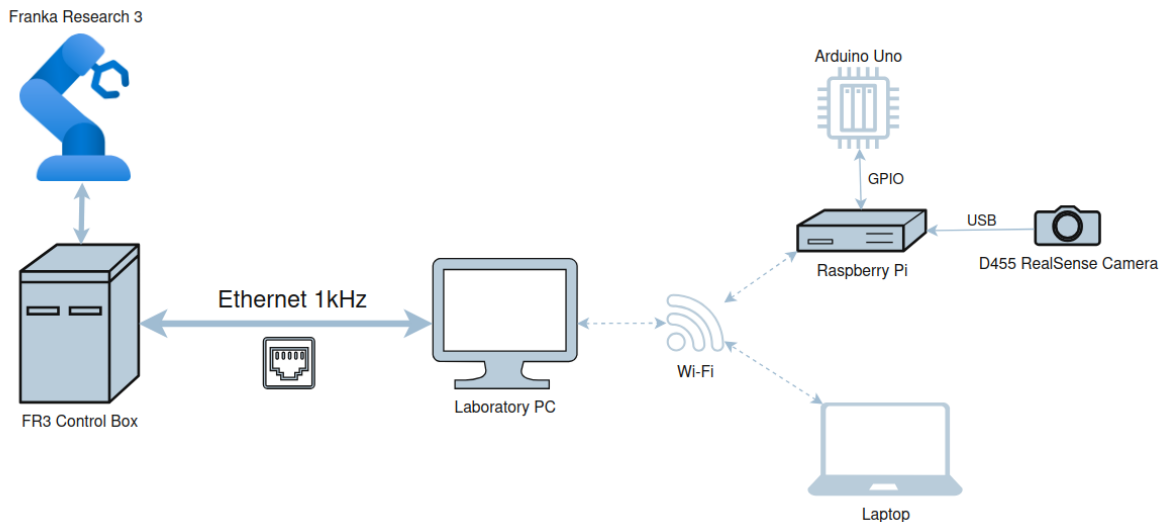


Figure 1: Setup

As visualized in the figure above, the major components are connected to each other either via ethernet or WI-FI. The Franka FR3 arm is connected directly to the Control Box. This control box is then connected via ethernet to the laboratory desktop PC managing the 1kHz real time communication with the robot controller. The rest of the system is split between the laptop and the Raspberry Pi which communicate with each other and with the desktop PC via WI-FI utilizing ROS2 Fast DDS for seamless communication between ROS2 nodes. There is also a hardware link between the Raspberry Pi and the Arduino UNO in which the two boards are sending signals to each other via a 3.3V <-> 5V logic shifter connected to their respective GPIO pins and a USB connection between the Raspberry Pi and the RealSense depth camera.

The software setup for the test is prepared by uploading the Arduino code for the communication test, preparing 2 VS Code windows, one with the local terminal and one connected to the Raspberry Pi via SSH and by launching the *franka_fr3_moveit_config* from the *franka_ros2* repository on the laboratory desktop PC.

Test Sheet

Technical information before commissioning:

Test form layout approved: (Mark with a cross)	Approved: X	Not Approved:
What to test:	Functionality of the supporting subsystems (gripper and RealSense camera) and the functionality of the full control pipeline	
Demonstration Responsible:	Wiktor Rutkowski	
Customer/tester:	Elling Ruud Øye	
Others present:	Project members: Wiktor Rutkowski Mert Can Sarikaya	
Date and time:	18.05.2026 13:30	
Test summary:	The test shall evaluate the functionality of each individual subsystem of the robot control pipeline as well as the functionality and usability of the full system by both testing small segments of the subsystems (f.eks. gripper <-> Raspberry Pi hardware communication) and performing larger demonstrations of functions of the system (f.eks. grasping by using a service call)	
Criteria for approval:	▫ Gripper functionality ▫ RealSense D400-series depth camera support ▫ Full robot URDF and SRDF models ▫ MoveIt wrapper functionality ▫ Motions package for creating new motions	
Result: (Mark with a cross)	Approved: X	Not Approved:

Test Execution

KUKA robot model test	Expected Results	Approved
Description: Open the KUKA robot description with the gripper attached and test it with simulated fake hardware		
In the local laptop terminal run the command: <i>ros2 launch kuka_kr_moveit_config moveit_planning_fake_hardware.launch.py</i>	rViz should launch with the motion planning plugin and the KUKA + gripper model loaded.	X
Change the goal position by dragging the end effector and execute the motion.	The robot should look and behave as expected, planning and executing the motion to the new goal pose.	X
Check for collision detection by setting a goal position which would result in a collision.	When detecting a collision, the relevant parts should turn red and the motion won't execute.	X
Additional Comments: Since it is impossible to test with the real hardware due to commissioning problems, the KUKA robot model is tested with simulated hardware, the rest of the system will be tested using the Franka fr3 manipulator.		

Gripper Communication Test	Expected Results	Approved
Description: Run the gripper communication Arduino code together with the ros2 node to verify that the devices can read and write to each other on the 2 connected pins.		
In the remote Raspberry Pi terminal run the <i>gripper_communication_test</i> node from the <i>gripper</i> package.	Messages logging the state of the Arduino and RPi pins should appear in the serial monitor output and the remote terminal. The registered state should match on them both.	X
Additional Comments:		

Gripper Driver Test	Expected Results	Approved
Description: Test the gripper subsystem functionality by calling the service to open/close the gripper.		
From the <i>Gripper_firmware</i> directory, open the <i>Gripper_firmware.ino</i> file and upload it to the Arduino.	The code uploads to Arduino without errors.	X
In the remote Raspberry Pi terminal run the <i>gripper_driver</i> node from the <i>gripper</i> package.	The gripper driver node starts without errors.	X
In the remote Raspberry Pi terminal run the <i>gripper_example</i> node from the <i>examples</i> package.	Logs about gripper open/close service calls should appear every 15 seconds. On gripper close calls the gripper should lock when both claws are closed, but when one or both claws are open the gripper should remain unlocked and an appropriate message should appear in the terminal	X
Additional Comments: This test is best performed with the Franka arm in hand guiding mode, this way the gripper could be manually moved onto the cylinder.		

RealSense Camera Test	Expected Results	Approved
Description: Test the RealSense depth camera subsystem by running a simple object detection script.		
In the remote Raspberry Pi terminal run the <i>camera_example</i> node from the <i>examples</i> package.	In the terminal running the node, logs should start appearing. After a short moment, a matplotlib plot showing the detected object clusters in 3D and 2D will should pop up on screen.	X
Additional Comments: While this object detection script is also available as a service, it is very basic and would likely need to be refactored in future development. With a bad robot position it is possible that the camera will not detect any objects, in such case, try moving the arm to a position which gives the camera a better view of the objects.		

MoveIt Wrapper Test	Expected Results	Approved
Description: Test the custom wrapper for the MoveIt package. In this test additional feedback about the intuitiveness of the wrapper should be gathered from the tester.		
In the remote Raspberry Pi terminal run the <i>franka_moveit_commander</i> node from the <i>moveit_commander</i> package.	The moveit commander node starts without errors.	X
In the remote Raspberry Pi terminal run the command: <i>ros2 action send_goal /goal_pose ri_seaweed_interfaces/action/MoveToPose "{target_pose: {header: {frame_id: 'fr3_link0'}, pose: {position: {x: 0.3898, y: 0.0, z: 0.5160}, orientation: {x: 0.0, y: 0.0, z: 0.70710678, w: 0.70710678}}}" --feedback</i>	Feedback from MoveIt group should start appearing in the terminal, and the robot should move to the pose specified in the command.	X
Try changing the position and orientation parameters of the command.	The robot should move to a different position. Setting a pose which would cause a collision should return MoveIt error code 4 and refuse to execute the motion.	X
In a code editor open the file: <i>ri_seaweed_ws/src/examples/examples/motion_example.py</i> Examine the code and run it as a node.	In the terminal running the node, logs should start appearing, and the robot should move through a series of waypoints.	X
Try to modify the waypoints of the robot.	Modifying the waypoints should be reflected by the changing robot motions.	X
Additional Comments:		

Full system demonstration	Expected Results	Approved
Description: Test and demonstrate the combined system functionality by running a fully automated cylinder pick up operation.		
In the remote Raspberry Pi terminal run the <i>return_home</i> from the <i>moveit_commander</i> package.	The robot should return to its home position.	X
In the remote Raspberry Pi terminal run the <i>auto_pickup_example</i> node from the <i>examples</i> package.	Logs should start appearing. The arm should move to a predefined position for image acquisition before moving to the approach pose for the cylinder. The arm should then lower onto the cylinder, lock the gripper and lift it up a couple of centimeters before dropping it and returning to the home position.	X
Additional Comments:		